DEPARTAMENTO DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo práctico 2: Implementación del TAD airMiles.

Implementación en Java del TAD airMiles: sistema de fidelización de una aerolinea.

---

9 de noviembre de 2026 — Algoritmos y Estructuras de Datos

Los4Bugueados

Integrante	LU	Correo electrónico
Campos, Felipe	356/20	felippe.1114@gmail.com
Portaluppi, Alejandro	51/19	alejandro_portaluppi@outlook.com
Chopite, Julio	617/24	juliodavidchopite@gmail.com
Rychter, Ariel	1294/21	arirychter@gmail.com

Facultad de Ciencias Exactas y Naturales

Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón I/Planta Baja)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Fax: (++54 +11) 4576-3300

http://www.exactas.uba.ar

1. Desarrollo y justificaciones.

Presentamos la propuesta de estructuras de datos que utilizamos para implementar el TP 2 explicando el propósito de cada una, cómo resolveríamos las operaciones y justificando sus complejidades.

1. Estructuras de datos

Utilizamos las siguientes estructuras:

- ABB de usuarios: ordenado por ID. Almacena objetos Usuario que contienen el ID, las millas totales, perdidas, disponibles, y un token (flag) para representar las relaciones de exclusividad entre transferidores. El ABB permite buscar e insertar usuarios en O(log U) en promedio y O(U) en el peor caso.

- Trie de promociones: almacena objetos Promocion, identificados por su nombre y con sus respectivos factores y topes. Observamos que la complejidad exigida para las operaciones que manipulan directamente a las promociones (iniciarPromocion, iniciarPromocionConTope y finalizarPromocion) es $O(|L|)$, siendo $|L|$ la longitud del nombre de una promoción. Esta curiosa solicitud nos llevó a concluir y utilizar la estructura Trie para manipular promociones, que cumple exitósamente dicha exigencia.

- Heap de Handles: es un MaxHeap de Handles que contienen el ID del usuario y sus millas disponibles, ordenado por la cantidad de millas disponibles. Lo utilizamos para obtener los usuarios con mayores millas disponibles y mantener el ranking. Las inserciones, extracciones y reajustes cuestan O(log U).

- ABB de Handles: Para buscar un elemento en O(log U), aprovechamos las propiedades del ABB para localizar rápidamente un usuario, obtener el handle y así tener la referencia de la posición del usuario en el heap y acceder a su posición en el array que representa al heap, así como a sus hijos y padre mediante las fórmulas dadas en clase en O(1).

- Variables auxiliares: elMasAcumulador y elMasCanjeador, que almacenan los IDs de los usuarios que corresponden a cada máximo histórico. Permiten resolver ambas consultas en O(1).

2. Procedimientos

Siguiendo el orden de la implementación, estas son las operaciones:

- AirMiles (constructor): inicializa el ABB, el Trie, el Heap y las variables auxiliares. También registra NoPromo en el Trie con factor 1 y sin tope. Como se trata de una clave fija ($|"NoPromo"| = 7$) su inserción tiene costo constante. Por lo tanto, la inicialización es O(1).

- registrarUsuario: crea un Usuario y lo inserta tanto en el ABB como en el heap. Si es el primer usuario registrado, inicializa los máximos históricos con su ID. Las inserciones cuestan O(log U) en promedio y O(U) en el peor caso, mientras que las demás operaciones cuestan O(1).

- registrarUsuarioConReferidor: realiza lo mismo que la operación anterior, pero además suma 500 millas totales y disponibles tanto al nuevo usuario como al referidor. Luego verifica si corresponde actualizar el mayor acumulador. Las búsquedas e inserciones en el ABB y en el heap mantienen una complejidad promedio O(log U) y O(U) en el peor caso.

- acumularMillas: busca al usuario en el ABB y la promoción en el Trie. Calcula las millas siguiendo la fórmula del enunciado, considerando el factor de categoría y el factor de promoción correspondiente. Si la promoción tiene un tope, actualiza el valor restante o la elimina si se agota. También verifica si corresponde actualizar al mayor acumulador y reajusta la posición del usuario en el heap. Las búsquedas cuestan O(log U) y $O(|L|)$, respectivamente, y el resto de las operaciones cuesta O(1). Al ser operaciones no anidadas, la complejidad total es $O(logU + |L|)$ en promedio y $O(U + |L|)$ en el peor caso.

- canjearMillas: busca al usuario en el ABB, actualiza sus millas disponibles y perdidas,

verifica si superó al mayor canjeador y reajusta su posición en el heap. La búsqueda en el ABB y el reajuste del heap cuestan O(log U) en promedio y O(U) en el peor caso.

- transferirMillas: busca al usuario origen y al destino en el ABB, actualiza sus saldos y las millas perdidas del origen, verifica si corresponde actualizar al mayor canjeador, modifica los tokens de exclusividad afectados si es necesario y reajusta las posiciones de ambos usuarios en el heap. Las búsquedas y actualizaciones mantienen una complejidad promedio O(log U) y O(U) en el peor caso.

- consultarSaldo: busca al usuario en el ABB y devuelve sus millas disponibles. Su complejidad es O(log U) en promedio y O(U) en el peor caso.

- consultarCategoria: busca al usuario en el ABB y determina su categoría a partir de las millas totales, comparándolas con los distintos rangos. Las comparaciones cuestan O(1), por lo que la complejidad es O(log U) en promedio y O(U) en el peor caso.

- rankingMillas: extrae sucesivamente los N máximos del heap y guarda sus IDs en un arreglo auxiliar. Como cada extracción devuelve el máximo restante, los usuarios quedan directamente ordenados de mayor a menor cantidad de millas. Luego los reinsertamos para conservar el heap original. Tanto las extracciones como las reinserciones cuestan O(log U), por lo que la complejidad total es O(N log U).

- iniciarPromocion: crea una promoción sin tope y la inserta en el Trie. Su complejidad es $O(|L|)$.

- iniciarPromocionConTope: crea una promoción con factor y tope, y la inserta en el Trie. Su complejidad es $O(|L|)$.

- finalizarPromocion: busca y elimina la promoción del Trie. Su complejidad es $O(|L|)$.

- elMasAcumulador y elMasCanjeador: devuelven directamente los IDs almacenados en las variables auxiliares. Ambas operaciones cuestan O(1).

- paresTransferidoresExclusivos: recorre el ABB y agrega los pares a un arreglo de resultados. Para evitar duplicados e invertir el orden de los pares, solo agrega aquellos en los que el ID del usuario sea mayor que el ID almacenado en su token de exclusividad. Como cada usuario se visita una sola vez y las operaciones realizadas durante el recorrido cuestan O(1), la complejidad total es O(U).

3. Observaciones

- NoPromo: la almacenamos en el Trie con factor 1 y sin tope. Como su nombre es fijo, su inserción durante la inicialización tiene costo constante.

- Ranking: Para obtener el ranking, las extracciones sucesivas ya devuelven los máximos ordenados. Luego reinsertamos los elementos para conservar el heap original. Para el ranking tenemos que hacer N veces desencolar, lo que hace desencolar es sacar el elemento de mayor prioridad que siempre es la raíz (el primero del arraylist), intercambia el ultimo elemento con el primero y reducimos en 1 el tamaño. Luego lo que se hace es el proceso de siftDown: si es menor que alguno de sus hijos se lo reemplaza por el mayor de sus hijos y se repite hasta que no se cumpla esta condición. Ademas nos interesa particularmente el encolar o insertar, donde agrandamos en 1 el tamaño del arraylist, insertamos el elemento al final del arraylist y hacemos siftUp: si el elemento es mayor que su padre, se hace swap con su padre, hasta que no se cumpla esta condición o hasta que llegue a ser la raíz y no se puede preguntar quien es su padre. Lo que da una complejidad de (N log U)

- También implementamos un Handle en lugar del atributo posicionEnHeap. La idea fue crear un ABB de handles donde el diccionario implementado del ABB tenga como clave un int que es un usuario y como valor handle(usuario).Para cada handle del ABB que tiene a su ID dentro, tiene una referencia de posición en el heap.De esta manera los usuarios no saben en qué estructura están dentro y no se rompe el encapsulamiento. Por otra parte el heap genera (y devuelve) a los handles.

- Pares de transferidores exclusivos: guardamos un token de exclusividad en cada usuario: 0 si todavía no participó en transferencias, -1 si fue exclusivo y dejó de serlo y, en otro caso, el ID de su par exclusivo (que son enteros positivos). La idea es que cuando dos usuarios realizan transferencias exclusivamente entre sí, sus tokens guarden mutuamente sus IDs. Si alguno realiza una transferencia con un tercer usuario, dejan de ser exclusivos: se actualizan a -1 los tokens de ambos usuarios de la relación anterior y también los de los usuarios que participan en la nueva transferencia. Para realizar estas actualizaciones, localizamos a las contrapartes mediante búsquedas en el ABB.